CLIENTOPS CRM — DOKUMENTACJA TECHNICZNA

Baza danych
Tabele & kolumny

Kompletny opis wszystkich tabel, kolumn, typów danych, flag i relacji w systemie ClientOps CRM. Dokument przeznaczony dla deweloperów pracujących nad backendem i migracjami.

WERSJA	DATA	BAZA DANYCH	LICZBA TABEL
1.0.0	Marzec 2026	PostgreSQL 16 + TimescaleDB + pgvector	55

Spis treści

WARSTWA 1 — TOŻSAMOŚĆ (3 TABELE)

- **users**
  Użytkownicy systemu
- **clients**
  Firmy / klienci
- **contacts**
  Osoby kontaktowe

WARSTWA 2 — SPRZEDAŻ (7 TABEL)

- **leads**
  Leady sprzedażowe
- **lead_activities**
  Historia kontaktów
- **offers**
  Oferty handlowe
- **lead_classifications**
  Scoring AI leadów (8 wymiarów)
- **discovery_workshops**
  Warsztaty Discovery (AI agent)
- **workshop_steps**
  Kroki warsztatu (9 etapów)
- **workshop_step_outputs**
  Wyjścia kroków warsztatu

WARSTWA 3 — REALIZACJA (45 TABEL)

- **projects**
  Projekty klientów
- **documents**
  Dokumenty TipTap
- **document_versions**
  Historia wersji
- **block_templates**
  Szablony bloków TipTap
- **block_template_scores**
  Scoring szablonów
- **sla_policies**
  Polityki SLA
- **tickets**
  Zgłoszenia klientów
- **ticket_comments**
  Komentarze do zgłoszeń
- **sla_incidents**
  Naruszenia SLA
- **kanban_boards**
  Tablice Kanban
- **kanban_columns**
  Kolumny tablic
- **kanban_cards**
  Karty Kanban
- **notifications**
  Powiadomienia
- **notification_preferences**
  Preferencje powiadomień
- **conversations**
  Wątki wiadomości
- **conversation_participants**
  Uczestnicy rozmów
- **messages**
  Wiadomości wewnętrzne
- **mentions**
  Oznaczenia @

- **activity_log**
  Audit trail (Spatie)
- **work_logs**
  Dziennik pracy
- **work_plan_items**
  Elementy planu pracy
- **seo_issues**
  Problemy SEO
- **seo_monthly_snapshots**
  Miesięczne snapshoty
- **seo_backlinks**
  Backlinki
- **seo_competitor_ranks**
  Rankingi konkurentów
- **seo_workshop_steps**
  SEO Workshop (4 etapy)
- **audits**
  Audyty zbiorcze
- **audit_scores**
  Wyniki kategorii
- **ux_audit_results**
  Wyniki audytu UX (axe-core)
- **knowledge_facts**
  Fakty (pgvector)
- **knowledge_entity_relations**
  Relacje encji
- **agent_knowledge**
  Wiedza agentów AI
- **slug_redirects**
  Przekierowania slugów
- **work_plans**
  Plany pracy
- **seo_analyses**
  Analizy SEO
- **seo_keyword_ranks**
  Rankingi słów kluczowych
- **seo_cwv_snapshots**
  Core Web Vitals
- **seo_competitors**
  Konkurenci
- **seo_keyword_opportunities**
  Szanse słów kluczowych
- **ai_visibility_scores**
  AI Visibility Score (8 czynników)
- **audit_issues**
  Problemy audytów
- **audit_reports**
  Raporty PDF audytów
- **security_audit_results**
  Wyniki audytu bezpieczeństwa
- **knowledge_entities**
  Encje wiedzy
- **knowledge_items**
  Drzewo wiedzy

WARSTWA 1

Tożsamość

Fundament systemu — kto istnieje w systemie. Użytkownicy (pracownicy agencji), klienci (firmy) i ich osoby kontaktowe.

users

Warstwa 1

Konta użytkowników systemu — pracownicy agencji zarządzający CRM-em. Obsługuje standardowe logowanie email+hasło, logowanie Google (Socialite) oraz opcjonalne 2FA przez email. Role przypisywane przez Spatie Permission.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator użytkownika.
name	varchar		Pełna nazwa wyświetlana (imię + nazwisko lub pseudonim). Używana w interfejsie i @wzmiankowaniach.
email	varchar	UNIQUE	Adres email — login do systemu. Musi być unikalny globalnie. Używany też do wysyłki kodów 2FA.
email_verified_at	timestamp	NULL	Znacznik czasu weryfikacji adresu email. null = email niezweryfikowany.
password	varchar	NULL	Hash hasła (bcrypt). null gdy użytkownik zarejestrował się wyłącznie przez Google OAuth — nie posiada hasła.
remember_token	varchar(100)	NULL	Token sesji „zapamiętaj mnie" — generowany przez Laravel automatycznie.
avatar_path	varchar	NULL	Ścieżka do avatara użytkownika (S3/MinIO przez Spatie Media Library). null = domyślny avatar.
phone	varchar	NULL	Numer telefonu — opcjonalny. Może być używany do kontaktu wewnętrznego.
is_active	boolean	BOOL	Czy konto jest aktywne. Nieaktywni użytkownicy nie mogą się zalogować — soft-disable bez usuwania danych.
two_factor_enabled	boolean	BOOL	Czy 2FA przez email jest włączone dla tego użytkownika. Włączane per-user z ustawień profilu.

KOLUMNA	TYP	FLAGI	OPIS
two_factor_code	varchar(6)	NULL	6-cyfrowy kod 2FA aktualnie oczekujący na weryfikację. Czyszczony po użyciu lub wygaśnięciu.
two_factor_expires_at	timestamp	NULL	Czas wygaśnięcia kodu 2FA. Kod ważny przez 10 minut od wygenerowania.
last_login_at	timestamp	NULL	Data i czas ostatniego udanego logowania. Przydatne do audytu bezpieczeństwa.
created_at	timestamp		Data utworzenia konta.
updated_at	timestamp		Data ostatniej modyfikacji rekordu.

RELACJE

- hasMany → `leads` (assigned_to)
- hasMany → `lead_activities`
- hasMany → `tickets` (created_by)
- hasMany → `ticket_comments`
- hasMany → `documents` (created_by)
- hasMany → `notification_preferences`
- morphMany → `activity_log` (causer)
- morphMany → `notifications`
- morphMany → `conversation_participants`
- morphMany → `messages` (sender)
- morphMany → `mentions` (mentioned)
- Spatie Permission: roles, permissions

clients

Warstwa 1

Firmy obsługiwane przez agencję — tenanci systemu. Każdy klient ma własną subdomenę (`{slug}.clientops.pl`) i izolowany widok danych przez Global Scope. Może powstać ręcznie lub przez konwersję leadu (status won).

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator klienta. Używany jako `client_id` we wszystkich tabelach warstwy 2 i 3 (multi-tenancy).
name	varchar		Pełna nazwa firmy np. „Acme Sp. z o.o.".
slug	varchar	UNIQUE	Unikalny identyfikator URL — globalnie unikatowy. Stanowi subdomenę panelu klienta (`acme.clientops.pl`). Immutable — zmiana nazwy firmy nie zmienia slugu. Generowany przez Spatie Sluggable.
nip	varchar	NULL	NIP firmy — opcjonalny, przydatny do fakturowania i integracji zewnętrznych.
address	varchar	NULL	Ulica i numer — część adresu pocztowego.

KOLUMNA	TYP	FLAGI	OPIS
city	varchar	NULL	Miasto siedziby firmy.
postal_code	varchar	NULL	Kod pocztowy.
country	varchar		Kraj siedziby (domyślnie „PL"). Istotny przy integracji z zewnętrznymi API i strefach czasowych.
website	varchar	NULL	Adres strony www — używany jako punkt startowy analiz SEO (crawler).
status	enum	ENUM	Stan relacji z klientem: `active` — aktywna współpraca, `inactive` — zawieszona, `archived` — zakończona (soft-archive).
sla_policy_id	bigint	FK NULL	Domyślna polityka SLA dla tego klienta → `sla_policies.id`. Dziedziczona przez nowe projekty i tickety, o ile nie nadpisana.
converted_from_lead_id	bigint	FK NULL	ID leadu, z którego powstał klient → `leads.id`. `null` = klient dodany ręcznie (nie przez konwersję). Umożliwia prześledzenie źródła klienta.
notes	text	NULL	Dowolne notatki wewnętrzne o kliencie — widoczne tylko dla pracowników agencji.
created_by	bigint	FK	Użytkownik, który utworzył rekord → `users.id`. Przydatne do audytu.
created_at	timestamp		Data dodania klienta do systemu.
updated_at	timestamp		Data ostatniej modyfikacji rekordu.

RELACJE

- hasMany → `contacts`
- hasMany → `leads`
- hasMany → `projects`
- hasMany → `offers`
- hasMany → `tickets`
- hasMany → `documents` (morph)
- belongsTo → `sla_policies`
- belongsTo → `leads` (converted_from)
- Multi-tenancy: `client_id` = tenant key

contacts

Warstwa 1

Osoby kontaktowe powiązane z firmą-klientem. Jedna firma może mieć wiele osób kontaktowych; jedna z nich może być oznaczona jako główna (`is_primary`). Kontakty mogą być powiązane z leadami jako reprezentanci firmy po stronie klienta.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator osoby kontaktowej.

KOLUMNA	TYP	FLAGI	OPIS
client_id	bigint	FK	Firma, do której należy kontakt → clients.id. Pełni rolę klucza multi-tenancy w tej tabeli.
first_name	varchar		Imię osoby kontaktowej.
last_name	varchar		Nazwisko osoby kontaktowej.
email	varchar	NULL	Adres email kontaktu. Opcjonalny — nie każda osoba musi mieć email (np. tylko telefon).
phone	varchar	NULL	Numer telefonu kontaktu. Opcjonalny.
position	varchar	NULL	Stanowisko w firmie np. „CEO", „Marketing Manager". Pomaga ustalić z kim rozmawiać w danym kontekście.
is_primary	boolean	BOOL	Czy to główna osoba kontaktowa w firmie. Używana jako domyślny adresat powiadomień i ofert. Każdy klient powinien mieć dokładnie jeden kontakt z true.
notes	text	NULL	Notatki wewnętrzne o osobie — widoczne tylko dla pracowników agencji.
created_at	timestamp		Data dodania kontaktu.
updated_at	timestamp		Data ostatniej modyfikacji rekordu.

RELACJE

belongsTo → clients

hasMany → leads (contact_id)

WARSTWA 2

Sprzedaż

Lejek sprzedażowy — od pierwszego kontaktu do podpisania umowy. Leady przechodzą przez 7 statusów Kanban; wygrany lead konwertuje do klienta. Discovery Workshop i kwalifikacja AI wspierają kwalifikację leadów. 7 tabel.

leads

Warstwa 2

Potencjalni klienci w procesie sprzedaży. Każdy lead przechodzi przez 7 statusów Kanban (new → `contacted` → `no_response` → `in_discussion` → `offer_sent` → won → `lost`). Status won automatycznie wyzwala konwersję do tabeli `clients`. Lead jest odrębnym bytem od klienta — zachowuje własną historię nawet po konwersji.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator leadu.
client_id	bigint	FK NULL	Wypełniane po konwersji leadu → `clients.id`. `null` = lead jeszcze nie skonwertowany. Pełni też rolę klucza multi-tenancy po konwersji.
contact_id	bigint	FK NULL	Konkretna osoba kontaktowa reprezentująca lead → `contacts.id`. Opcjonalna — lead może nie mieć jeszcze przypisanego kontaktu.
assigned_to	bigint	FK NULL	Pracownik odpowiedzialny za lead → `users.id`. Używany w Policy przy sprawdzaniu dostępu.
title	varchar		Tytuł / nazwa leadu np. „Strona www + SEO — Kowalski Transport". Widoczny na karcie Kanban.
company_name	varchar	NULL	Nazwa firmy potencjalnego klienta — zanim powstanie pełny rekord `clients`. Kopiowana do `clients.name` przy konwersji.
slug	varchar		Identyfikator URL leadu, unikalny per `client_id`. Immutable — Spatie Sluggable z `onUpdate = false`.
status	enum	ENUM	Aktualny etap w lejku: new | `contacted` | `no_response` | `in_discussion` | `offer_sent` | won | `lost`. PHP enum `LeadStatus`. Zmiana statusu przesuwa kartę Kanban przez event→listener.
source	enum	ENUM	Skąd pochodzi lead: `manual` — dodany ręcznie, `import` — CSV, `form` — formularz kontaktowy, `referral` — polecenie.

KOLUMNA	TYP	FLAGI	OPIS
value	decimal	NULL	Szacowana wartość kontraktu (PLN). Używana w raportach pipeline i prognozowaniu przychodów.
expected_close_date	date	NULL	Przewidywana data zamknięcia sprzedaży. Używana w SLA i raportach pipeline.
notes	text	NULL	Dodatkowe notatki wewnętrzne — widoczne tylko dla pracowników.
converted_at	timestamp	NULL	Data i czas konwersji leadu do klienta. `null` = niekonwertowany. Wypełniane automatycznie przy ustawieniu statusu `won`.
converted_to_client_id	bigint	FK NULL	Referencja do utworzonego klienta → `clients.id`. Umożliwia przejście z widoku leadu bezpośrednio do profilu klienta.
created_by	bigint	FK	Pracownik który dodał lead → `users.id`.
created_at	timestamp		Data dodania leadu.
updated_at	timestamp		Data ostatniej modyfikacji.

RELACJE

belongsTo → **clients** (po konwersji)
belongsTo → **contacts**
belongsTo → **users** (assigned_to)
hasMany → **lead_activities**
hasMany → **offers**
hasOne → **kanban_cards** (morph)
hasOne → **clients** (converted_to)

lead_activities

Warstwa 2

Chronologiczna historia wszystkich kontaktów i zdarzeń związanych z leadem. Każda akcja pracownika (telefon, email, spotkanie, notatka) jest rejestrowana jako oddzielny rekord. Stanowi pełny audit trail procesu sprzedaży.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator aktywności.
lead_id	bigint	FK	Lead, którego dotyczy aktywność → `leads.id`.
user_id	bigint	FK	Pracownik który wykonał czynność → `users.id`. Przy automatycznych wpisach (np. `status_change`) może być systemowy user.
type	enum	ENUM	Typ aktywności: `call` — rozmowa telefoniczna, `email` — wysłany/odebrany email, `meeting` — spotkanie, `note` — notatka tekstowa, `status_change` — automatyczny wpis przy zmianie statusu leadu.

KOLUMNA	TYP	FLAGI	OPIS
description	text	NULL	Treść notatki / opis wykonanej czynności. Dla `status_change` zawiera informację o poprzednim i nowym statusie.
occurred_at	timestamp		Rzeczywisty czas wykonania czynności — może różnić się od `created_at` gdy pracownik loguje aktywność z przeszłości (np. spotkanie sprzed 2 dni).
metadata	json	JSON NULL	Dodatkowe dane specyficzne dla typu: dla `status_change` — `{"from": "contacted", "to": "in_discussion"}`; dla `call` — czas trwania; dla `meeting` — lokalizacja.
created_at	timestamp		Data dodania wpisu do systemu (może różnić się od `occurred_at`).

RELACJE

belongsTo → **leads**

belongsTo → **users**

offers

Warstwa 2

Oferty handlowe wysyłane do leadów. Oferta jest odrębnym bytem — nie aliasem leadu ani klienta. Może zawierać opcjonalny dokument TipTap z pełną treścią oferty. Akceptacja oferty poprzedza konwersję leadu do klienta.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator oferty.
lead_id	bigint	FK	Lead, dla którego przygotowano ofertę → `leads.id`. Jeden lead może mieć wiele ofert (np. różne warianty cenowe).
client_id	bigint	FK NULL	Wypełniane automatycznie przy konwersji leadu → `clients.id`. `null` = oferta dla jeszcze nieskonwertowanego leadu. Pełni rolę klucza multi-tenancy po konwersji.
title	varchar		Tytuł oferty np. „Pakiet SEO Premium — Q1 2026".
slug	varchar		Identyfikator URL oferty, unikalny per `client_id`. Immutable.
status	enum	ENUM	Aktualny stan oferty: `draft` — w przygotowaniu, `sent` — wysłana do klienta, `accepted` — zaakceptowana, `rejected` — odrzucona, `expired` — minął termin ważności.
value	decimal	NULL	Wartość oferty netto (PLN). Może różnić się od szacowanej wartości leadu.

KOLUMNA	TYP	FLAGI	OPIS
valid_until	date	NULL	Termin ważności oferty. Po przekroczeniu tej daty status zmienia się automatycznie na `expired` (scheduled job).
document_id	bigint	FK NULL	Opcjonalny pełny dokument oferty stworzony w edytorze TipTap → `documents.id`. Może być eksportowany do PDF przez Browsershot.
notes	text	NULL	Wewnętrzne notatki dotyczące oferty — niewidoczne dla klienta.
accepted_at	timestamp	NULL	Data i czas akceptacji oferty przez klienta.
rejected_at	timestamp	NULL	Data i czas odrzucenia oferty.
converted_to_client_at	timestamp	NULL	Data, kiedy lead powiązany z ofertą został skonwertowany do klienta. Pozwala mierzyć czas od oferty do zamknięcia sprzedaży.
public_token	uuid	NULL	Token UUID do publicznego podglądu oferty bez logowania. Generowany przy wysyłce (status: sent). Link: `/oferta/{public_token}`. Może być unieważniony przez usunięcie tokenu.
public_expires_at	timestamp	NULL	Opcjonalna data wygaśnięcia publicznego linku. Po tej dacie link przestaje działać niezależnie od statusu oferty. `null` = link aktywny do ręcznego unieważnienia.
created_by	bigint	FK	Pracownik który stworzył ofertę → `users.id`.
created_at	timestamp		Data utworzenia oferty.
updated_at	timestamp		Data ostatniej modyfikacji.

RELACJE

- belongsTo → **leads**
- belongsTo → **clients** (po konwersji)
- belongsTo → **documents** (opcjonalnie)
- belongsTo → **users** (created_by)

lead_classifications

Warstwa 2

8-wymiarowy scoring kwalifikacyjny leadu generowany przez AI po zakończeniu Discovery Workshop lub uzupełniany ręcznie przez managera. Każdy wymiar oceniany w skali 1–10. Wynik ogólny to ważona średnia wszystkich wymiarów. Jeden rekord per lead (unique constraint).

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator rekordu klasyfikacji.
lead_id	bigint	FK	Lead którego dotyczy klasyfikacja → `leads.id`. Unique — jeden rekord per lead.
budget_score	tinyint		Budżet klienta (1–10). 10 = budżet w pełni zgodny z ofertą agencji.
timeline_score	tinyint		Zgodność terminów (1–10). 10 = termin realizacji realistyczny i zgodny z zasobami.
decision_score	tinyint		Decyzyjność osoby kontaktowej (1–10). 10 = kontakt jest ostatecznym decydentem.
problem_score	tinyint		Jasność problemu do rozwiązania (1–10). 10 = klient precyzyjnie definiuje problem.
fit_score	tinyint		Dopasowanie klienta do profilu idealnego klienta agencji (1–10).
competition_score	tinyint		Poziom konkurencji w procesie zakupowym (1–10). 10 = agencja jedynym wyborem.
urgency_score	tinyint		Pilność potrzeby klienta (1–10). 10 = klient potrzebuje natychmiastowego działania.
ltv_score	tinyint		Potencjalna wartość długoterminowa (Lifetime Value) klienta (1–10).
overall_score	decimal(4,2)		Ważona średnia wszystkich 8 wymiarów. Zakres 1.00–10.00. Używana do sortowania leadów w pipeline.
ai_reasoning	text	NULL	Uzasadnienie scoringu wygenerowane przez AI (Claude). Opis mocnych i słabych stron leadu.
classified_at	timestamp		Data i czas przeprowadzenia klasyfikacji.

RELACJE

belongsTo → `leads` (unique — 1:1)

discovery_workshops

Warstwa 2

Warsztaty Discovery prowadzone przez AI agenta "Kasia" (Claude Sonnet). 9-krokowy wywiad który pozwala agencji zrozumieć potrzeby klienta przed stworzeniem oferty.

Wyniki warsztatu są podstawą dla klasyfikacji AI leadu (`lead_classifications`). Jeden lead może mieć wiele warsztatów (np. ponowne przeprowadzenie po czasie).

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator warsztatu.
lead_id	bigint	FK	Lead dla którego prowadzony jest warsztat → `leads.id`.
conducted_by	bigint	FK	Pracownik agencji który zainicjował warsztat → `users.id`. Agent AI prowadzi wywiad, ale człowiek inicjuje sesję.
status	enum	ENUM	`in_progress` — warsztat trwa, `completed` — wszystkie 9 kroków zakończone, `abandoned` — przerwany przed ukończeniem.
started_at	timestamp		Data i czas rozpoczęcia warsztatu.
completed_at	timestamp	NULL	Data i czas ukończenia warsztatu (wszystkie kroki zakończone). `null` = warsztat niezakończony.

RELACJE

belongsTo → **leads** | belongsTo → **users** (conducted_by) | hasMany → **workshop_steps**

workshop_steps

Warstwa 2

Pojedynczy krok warsztatu Discovery (1 z 9). Każdy krok ma przypisany prompt systemowy dla agenta AI oraz przechowuje stan (aktywny/zakończony/pominięty). Kolejność kroków: cel projektu → obecna sytuacja → problemy → budżet → timeline → decyzyjność → konkurencja → oczekiwania → podsumowanie.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator kroku.
workshop_id	bigint	FK	Warsztat do którego należy krok → `discovery_workshops.id`.
step_number	tinyint		Numer kroku (1–9). Determinuje kolejność w wywiadzie.
title	varchar		Tytuł kroku np. "Cel projektu", "Budżet i timeline".
status	enum	ENUM	`pending` — oczekuje, `active` — aktualnie prowadzony przez agenta, `completed` — zakończony, `skipped` — pominięty przez użytkownika.
agent_prompt	text		Prompt systemowy dla agenta AI "Kasia" specyficzny dla tego kroku. Determinuje jakie pytania zadaje agent.
completed_at	timestamp	NULL	Data i czas ukończenia kroku.

RELACJE

belongsTo → `discovery_workshops` hasMany → `workshop_step_outputs`

workshop_step_outputs

Warstwa 2

Wyjścia z pojedynczego kroku warsztatu — odpowiedzi agenta AI, inputy użytkownika i podsumowania. Jeden krok może mieć wiele wpisów (wielokrotna wymiana w ramach kroku). Przesyłane przez SSE (Server-Sent Events) w czasie rzeczywistym do frontendu.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator wyjścia.
step_id	bigint	FK	Krok warsztatu do którego należy wyjście → workshop_steps.id.
output_type	enum	ENUM	agent_response — odpowiedź AI "Kasia" (streaming), user_input — wiadomość od użytkownika, summary — podsumowanie kroku generowane przez AI.
content	text		Treść wyjścia. Dla agent_response — pełna odpowiedź po zakończeniu streamingu. Dla summary — wyekstrahowane kluczowe informacje z kroku.
created_at	timestamp		Data i czas utworzenia wyjścia.

RELACJE

belongsTo → `workshop_steps`

WARSTWA 3

Realizacja

Właściwa praca dla klientów — projekty, dokumenty, zgłoszenia, SLA, SEO, audyty, komunikacja i logi. 45 tabel.

projects

Warstwa 3

Realizacje usług dla klientów — centrum Warstwy 3. Większość zasobów systemu (`documents`, `seo_analyses`, `audits`, `tickets`) jest przypisana do projektu. Projekt dziedziczy politykę SLA od klienta, ale może mieć własną nadpisaną.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator projektu.
client_id	bigint	FK	Klient właściciel projektu → `clients.id`. Klucz multi-tenancy — Global Scope filtruje po tym polu.
name	varchar		Nazwa projektu np. „SEO + Content Marketing Q1 2026".
slug	varchar		Identyfikator URL projektu, unikalny per `client_id`. Immutable. Używany w URL panelu klienta.
description	text	NULL	Opis zakresu projektu — widoczny dla klienta w jego panelu.
status	enum	ENUM	Stan projektu: `planning` — w planowaniu, `active` — w realizacji, `on_hold` — wstrzymany, `completed` — zakończony, `cancelled` — anulowany.
sla_policy_id	bigint	FK NULL	Polityka SLA dedykowana temu projektowi → `sla_policies.id`. `null` = dziedziczy politykę z `clients.sla_policy_id`.
assigned_to	bigint	FK NULL	Główny opiekun projektu → `users.id`. Sprawdzany przez Policy przy dostępie managera/pracownika.
deadline	date	NULL	Termin realizacji projektu. Przekroczenie deadlinu bez ukończenia wyzwala `sla_incidents` z `breach_type = deadline`.
started_at	timestamp	NULL	Data faktycznego rozpoczęcia prac. Wypełniana przy zmianie statusu na `active`.
completed_at	timestamp	NULL	Data zakończenia projektu. Wypełniana przy statusie `completed`.
created_by	bigint	FK	Pracownik który utworzył projekt → `users.id`.
created_at	timestamp		Data utworzenia projektu.

KOLUMNA	TYP	FLAGI	OPIS
updated_at	timestamp		Data ostatniej modyfikacji.

RELACJE

- belongsTo → **clients**
- belongsTo → **sla_policies**
- belongsTo → **users** (assigned_to)
- hasMany → **documents** (morph)
- hasMany → **seo_analyses**
- hasMany → **audits**
- hasMany → **tickets**
- hasMany → **seo_competitors**
- morphMany → **sla_incidents** (deadline breach)

documents

Warstwa 3

Dokumenty tworzone w edytorze TipTap (ProseMirror). Polimorficznie powiązane z projektami, klientami lub ticketami. Przechowują treść w formacie JSON (TipTap) i wyrenderowanym HTML. Mogą być eksportowane do PDF przez Browsershot (async w kolejce).

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator dokumentu.
documentable_type	varchar	POLY	Klasa modelu właściciela dokumentu: `Project` | `Client` | `Ticket`. Para z `documentable_id` tworzy relację polimorficzną.
documentable_id	bigint	POLY	ID rekordu właściciela — projekt, klient lub ticket.
title	varchar		Tytuł dokumentu widoczny na liście. Używany też do generowania slugu.
slug	varchar		Identyfikator URL dokumentu, unikalny per `documentable`. Immutable — Spatie Sluggable.
content_json	longtext	JSON NULL	Treść dokumentu w formacie JSON TipTap (ProseMirror document model). To jest źródło prawdy — używane do edycji.
content_html	longtext	NULL	Wyrenderowany HTML — generowany z `content_json` przy zapisie. Używany do podglądu i eksportu PDF przez Browsershot.
pdf_path	varchar	NULL	Ścieżka do wygenerowanego pliku PDF w S3/MinIO. `null` = PDF nie został jeszcze wygenerowany lub jest w kolejce. Generowanie asynchroniczne przez Browsershot.
version	int		Aktualny numer wersji dokumentu. Inkrementowany przy każdym zapisie. Poprzednie wersje przechowywane w `document_versions`.
is_published	boolean	BOOL	Czy dokument jest widoczny dla klienta w jego panelu. `false` = roboczy draft, widoczny tylko dla

KOLUMNA	TYP	FLAGI	OPIS
			pracowników.
created_by	bigint	FK	Pracownik który stworzył dokument → users.id.
created_at	timestamp		Data utworzenia dokumentu.
updated_at	timestamp		Data ostatniej modyfikacji.

RELACJE

morphTo → projects / clients / tickets

hasMany → document_versions

belongsTo → users (created_by)

hasOne ← offers (document_id)

document_versions

Warstwa 3

Historia wersji dokumentów TipTap. Przy każdym zapisie bieżąca treść jest archiwizowana tutaj przed nadpisaniem. Umożliwia przeglądanie i przywracanie wcześniejszych wersji.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator wersji.
document_id	bigint	FK	Dokument, do którego należy ta wersja → documents.id.
version	int		Numer wersji — odpowiada wartości documents.version w chwili archiwizacji. Para (document_id, version) jest unikalna.
content_json	longtext	JSON	Zarchiwizowana treść dokumentu w formacie JSON TipTap — snapshot stanu przed nadpisaniem.
title	varchar		Tytuł dokumentu w chwili archiwizacji — może różnić się od aktualnego tytułu.
created_by	bigint	FK	Użytkownik który dokonał zapisu tworzącego tę wersję → users.id.
created_at	timestamp		Data archiwizacji wersji.

RELACJE

belongsTo → documents

belongsTo → users (created_by)

sla_policies

Warstwa 3

Polityki SLA definiujące wymagane czasy reakcji i rozwiązania zgłoszeń. Mogą być globalne (client_id = null) lub dedykowane konkretnemu klientowi. Polityka

klienta nadpisuje globalną; polityka projektu nadpisuje politykę klienta.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator polityki SLA.
client_id	bigint	FK NULL	Klient, dla którego dedykowana jest polityka → `clients.id`. `null` = polityka globalna (domyślna dla wszystkich klientów bez własnej polityki).
name	varchar		Nazwa polityki np. „Standard", „Premium 24/7", „Enterprise".
priority	enum	ENUM	Poziom priorytetu zgłoszeń, dla których obowiązuje ta polityka: `low` | `medium` | `high` | `critical`. Ticket dziedziczy politykę odpowiadającą swojemu priorytetowi.
response_time_hours	int		Maksymalny czas pierwszej reakcji na zgłoszenie (w godzinach). Przekroczenie → `sla_incidents` z `breach_type = response`.
resolution_time_hours	int		Maksymalny czas rozwiązania zgłoszenia (w godzinach). Przekroczenie → `sla_incidents` z `breach_type = resolution`.
business_hours_only	boolean	BOOL	Czy licznik SLA uwzględnia tylko godziny robocze (pon-pt 8:00–17:00). `true` = weekendy i noce nie są wliczane. Istotne przy automatycznym obliczaniu `sla_response_due_at`.
created_at	timestamp		Data utworzenia polityki.
updated_at	timestamp		Data ostatniej modyfikacji.

RELACJE

- belongsTo → **clients** (opcjonalnie)
- hasMany ← **clients** (sla_policy_id)
- hasMany ← **projects** (sla_policy_id)
- hasMany ← **tickets** (sla_policy_id)

tickets

Warstwa 3

Zgłoszenia serwisowe tworzone przez klientów. Sfera ludzka — klient ręcznie zgłasza problem lub pytanie. Każde zgłoszenie ma własne liczniki SLA (`sla_response_due_at`, `sla_resolution_due_at`) obliczane na podstawie przypisanej polityki SLA i priorytetu. Może mieć kartę na tablicy Kanban.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Numeryczny identyfikator zgłoszenia — wyświetlany jako #142 w interfejsie. Nie posiada slugu.
client_id	bigint	FK	Klient który złożył zgłoszenie → `clients.id`. Klucz multi-tenancy.
project_id	bigint	FK	Projekt, którego dotyczy zgłoszenie → `projects.id`. Zgłoszenia są zawsze powiązane z konkretnym projektem.
created_by	bigint	POLY	ID autora zgłoszenia. Para z `created_by_type` — może to być `User` (pracownik zakłada ticket za klienta) lub `Client` (klient z panelu).
created_by_type	varchar	POLY	Klasa autora: `User` | `Client`.
title	varchar		Tytuł zgłoszenia — krótki opis problemu.
description	text		Pełny opis problemu podany przez zgłaszającego.
priority	enum	ENUM	Priorytet zgłoszenia: `low` | `medium` | `high` | `critical`. Determinuje, która polityka SLA zostanie zastosowana do obliczenia terminów.
status	enum	ENUM	Aktualny stan obsługi: `new` → `open` → `in_progress` → `waiting` → `resolved` → `closed`. Zmiana statusu przesuwa kartę Kanban.
sla_policy_id	bigint	FK NULL	Polityka SLA zastosowana do tego ticketu → `sla_policies.id`. Ustalana przy tworzeniu na podstawie priorytetu i klienta/projektu.
sla_response_due_at	timestamp	NULL	Deadline pierwszej odpowiedzi — obliczany jako `created_at + response_time_hours` (z uwzględnieniem godzin roboczych jeśli `business_hours_only = true`).
sla_resolution_due_at	timestamp	NULL	Deadline rozwiązania zgłoszenia — obliczany jako `created_at + resolution_time_hours`.

KOLUMNA	TYP	FLAGI	OPIS
first_responded_at	timestamp	NULL	Czas pierwszej odpowiedzi pracownika. Wypełniany automatycznie przy pierwszym komentarzu non-internal. Używany do sprawdzenia czy SLA response zostało dotrzymane.
resolved_at	timestamp	NULL	Czas zmiany statusu na `resolved`. Używany do pomiaru dotrzymania SLA resolution.
closed_at	timestamp	NULL	Czas zamknięcia zgłoszenia (status `closed`). Ticket zamknięty nie może być ponownie otwarty — wymaga nowego zgłoszenia.
kanban_card_id	bigint	FK NULL	Powiązana karta Kanban → `kanban_cards.id`. Umożliwia zarządzanie ticketami z poziomu tablicy Kanban.
created_at	timestamp		Data złożenia zgłoszenia. Punkt startowy liczników SLA.
updated_at	timestamp		Data ostatniej modyfikacji.

RELACJE

- belongsTo → **clients**
- belongsTo → **projects**
- belongsTo → **sla_policies**
- belongsTo → **kanban_cards**
- hasMany → **ticket_comments**
- hasMany → **documents** (morph)
- morphMany → **sla_incidents**

ticket_comments

Warstwa 3

Komentarze do zgłoszeń — widoczne dla klienta lub wyłącznie wewnętrzne (notatki między pracownikami). Pierwszy komentarz non-internal od pracownika automatycznie wypełnia `tickets.first_responded_at`.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator komentarza.
ticket_id	bigint	FK	Zgłoszenie, do którego należy komentarz → `tickets.id`.
author_id	bigint	POLY	ID autora komentarza. Para z `author_type`.
author_type	varchar	POLY	Klasa autora: `User` (pracownik) | `Client` (klient odpowiada z panelu). Decyduje skąd pobierać dane autora.
is_internal	boolean	BOOL	Czy komentarz jest wewnętrzną notatką. `true` = widoczny tylko dla pracowników agencji, klient go nie zobaczy. `false` = publiczny, widoczny dla klienta w panelu.

KOLUMNA	TYP	FLAGI	OPIS
content	text		Treść komentarza. Może zawierać @wzmianki — parser tworzy wpisy w tabeli `mentions`.
created_at	timestamp		Data dodania komentarza.
updated_at	timestamp		Data ostatniej edycji komentarza.

RELACJE

belongsTo → `tickets` | morphTo → `users` / `clients` (author) | morphMany → `mentions`

sla_incidents

Warstwa 3

Naruszenia SLA wykrywane automatycznie przez `CheckSlaBreachesJob` uruchamiany co 5 minut. Sfera systemowa — tworzony przez scheduler, nie przez człowieka. Widoczny tylko dla adminów i managerów. Jeden ticket może wygenerować wiele incydentów (`response` breach, potem `resolution` breach).

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator incydentu SLA.
subject_type	varchar	POLY	Klasa obiektu naruszającego SLA: `Ticket` (przekroczony czas reakcji/rozwiązania) | `Project` (przekroczony deadline). Para z `subject_id`.
subject_id	bigint	POLY	ID ticketu lub projektu naruszającego SLA.
breach_type	enum	ENUM	Typ naruszenia: `response` — brak pierwszej odpowiedzi w terminie, `resolution` — brak rozwiązania w terminie, `deadline` — przekroczony deadline projektu.
breached_at	timestamp		Czas, kiedy SLA zostało naruszone (moment przekroczenia terminu).
detected_at	timestamp		Czas wykrycia naruszenia przez `CheckSlaBreachesJob`. Może być późniejszy niż `breached_at` o maksymalnie 5 minut (interwał joba).
notified	boolean	BOOL	Czy powiadomienie o naruszeniu zostało wysłane. Zapobiega wielokrotnemu wysyłaniu powiadomień dla tego samego incydentu.
acknowledged_by	bigint	FK NULL	Pracownik który potwierdził przyjęcie incydentu do wiadomości → `users.id`. `null` = incydent niepotwierdzony.
acknowledged_at	timestamp	NULL	Data i czas potwierdzenia incydentu.
resolved_at	timestamp	NULL	Data i czas rozwiązania incydentu (zgłoszenie zostało obsłużone lub projekt ukończony po naruszeniu).

KOLUMNA	TYP	FLAGI	OPIS
created_at	timestamp		Data utworzenia rekordu przez scheduler.

RELACJE

morphTo → **tickets** / **projects** (subject)

belongsTo → **users** (acknowledged_by)

kanban_boards

Warstwa 3

Tablice Kanban — kontenery dla zestawów kolumn. System obsługuje trzy typy tablic: dla leadów, ticketów i zadań projektowych. Ujednolicony model pozwala używać jednego komponentu React dla wszystkich typów.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator tablicy.
name	varchar		Nazwa tablicy np. „Leady", „Zgłoszenia serwisowe".
board_type	enum	ENUM	Typ tablicy: `leads` | `tickets` | `project_tasks`. Determinuje jakiego typu karty (`cardable_type`) mogą znajdować się na tablicy.
created_at	timestamp		Data utworzenia tablicy.

RELACJE

hasMany → **kanban_columns**

hasMany → **kanban_cards**

kanban_columns

Warstwa 3

Kolumny w tablicy Kanban. Dla tablicy leadów predefiniowane 7 kolumn odpowiada wartościom enuma `LeadStatus`. Kolejność kolumn określa pole `position`.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator kolumny.
board_id	bigint	FK	Tablica, do której należy kolumna → `kanban_boards.id`.
name	varchar		Wyświetlana nazwa kolumny np. „Nowy", „W realizacji", „Wygrany".
position	int		Kolejność wyświetlania kolumny od lewej (0-based). Przy drag & drop kolumn aktualizowane przez dnd-kit.
color	varchar	NULL	Kolor nagłówka kolumny (hex lub nazwa CSS). Używany do wizualnego rozróżnienia kolumn.

KOLUMNA	TYP	FLAGI	OPIS
created_at	timestamp		Data utworzenia kolumny.

RELACJE

belongsTo → kanban_boards

hasMany → kanban_cards

kanban_cards

Warstwa 3

Karty Kanban — polimorficzne wrappery dla leadów, ticketów lub zadań projektowych. Zmiana statusu leadu automatycznie przesuwa kartę do odpowiedniej kolumny przez event → listener. Pozycja karty w kolumnie aktualizowana przy drag & drop.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator karty.
board_id	bigint	FK	Tablica, na której znajduje się karta → `kanban_boards.id`.
column_id	bigint	FK	Kolumna, w której aktualnie znajduje się karta → `kanban_columns.id`. Aktualizowane przy zmianie statusu lub drag & drop.
position	int		Kolejność karty w kolumnie (0-based). Aktualizowane przez dnd-kit przy przeciąganiu.
cardable_type	varchar	POLY	Klasa modelu reprezentowanego przez kartę: Lead | `Ticket` | `ProjectTask`.
cardable_id	bigint	POLY	ID leadu, ticketu lub zadania powiązanego z kartą.
assigned_to	bigint	FK NULL	Pracownik przypisany do karty → `users.id`. Wyświetlany jako avatar na karcie Kanban.
due_at	timestamp	NULL	Termin wykonania widoczny na karcie. Wyświetlany w kolorze czerwonym po przekroczeniu.
created_at	timestamp		Data utworzenia karty.
updated_at	timestamp		Data ostatniej modyfikacji.

RELACJE

belongsTo → kanban_boards

belongsTo → kanban_columns

morphTo → leads / tickets / ProjectTask

belongsTo → users (assigned_to)

notifications

Warstwa 3

Powiadomienia in-app generowane przez Laravel Notifications. Standardowa tabela frameworka — każde powiadomienie to rekord JSON z typem i danymi. Odczytywane przez frontend przez Laravel Echo / Reverb (real-time broadcast). Stanowią też inbox wewnętrzny — łączą powiadomienia systemowe z wiadomościami od ludzi.

KOLUMNA	TYP	FLAGI	OPIS
id	uuid	PK	UUID powiadomienia — standard Laravel Notifications (nie bigint).
type	varchar		Pełna nazwa klasy powiadomienia np. `App\Notifications\TicketUpdated`. Używana do renderowania odpowiedniego komponentu w UI.
notifiable_type	varchar	POLY	Klasa odbiorcy powiadomienia: `User` | `Client`.
notifiable_id	bigint	POLY	ID odbiorcy.
data	json	JSON	Payload powiadomienia — tytuł, treść, link, ID powiązanego obiektu. Struktura zależy od `type`. Renderowany przez frontend na podstawie pola `type`.
read_at	timestamp	NULL	Czas odczytania powiadomienia. `null` = nieprzeczytane. Używany do wyświetlania licznika nieprzeczytanych w UI.
created_at	timestamp		Data wysłania powiadomienia.

RELACJE

morphTo → **users** / **clients** (notifiable)

notification_preferences

Warstwa 3

Preferencje użytkownika dotyczące kanałów powiadomień per typ zdarzenia. Jeden rekord = jeden typ zdarzenia dla jednego użytkownika. Pozwala np. ustawić, że naruszenia SLA przychodzą emailem i SMS-em, ale wiadomości — tylko in-app. SMS respektuje quiet hours.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator preferencji.
user_id	bigint	FK	Użytkownik, którego dotyczą preferencje → `users.id`.
notification_type	varchar		Typ zdarzenia: `ticket_update` | `sla_breach` | `audit_complete` | `pdf_ready` | `deadline` | `mention` | `message` | `task_assigned` | `comment_reply`.

KOLUMNA	TYP	FLAGI	OPIS
channels	`json`	JSON	Tablica aktywnych kanałów dla tego typu zdarzenia np. `["mail", "database", "broadcast"]`. Dostępne kanały: `mail` | `sms` | `database` | `broadcast`.
quiet_hours_from	`time`	NULL	Początek ciszy nocnej (np. `22:00`). W tym przedziale kanał SMS jest blokowany.
quiet_hours_to	`time`	NULL	Koniec ciszy nocnej (np. `08:00`).
quiet_hours_timezone	`varchar`		Strefa czasowa użytkownika dla quiet hours np. `Europe/Warsaw`. Niezbędna do poprawnego przeliczenia czasu.
created_at	`timestamp`		Data utworzenia preferencji.
updated_at	`timestamp`		Data ostatniej modyfikacji.

RELACJE

belongsTo → **users**

conversations

Warstwa 3

Wątki wiadomości wewnętrznych — zunifikowany inbox łączący komunikację człowiek→człowiek z notyfikacjami systemowymi. Rozmowa może być między pracownikami lub między pracownikiem a klientem. Każda nowa wiadomość tworzy powiadomienie dla wszystkich uczestników przez Reverb.

KOLUMNA	TYP	FLAGI	OPIS
id	`bigint`	PK	Unikalny identyfikator rozmowy.
created_by	`bigint`	FK	Użytkownik który zainicjował rozmowę → `users.id`.
subject	`varchar`	NULL	Opcjonalny temat rozmowy. `null` = rozmowa bez tematu (np. szybka wiadomość direct).
created_at	`timestamp`		Data rozpoczęcia rozmowy.

RELACJE

belongsTo → **users** (created_by)

hasMany → **conversation_participants**

hasMany → **messages**

conversation_participants

Warstwa 3 | PIVOT

Pivot łączący rozmowy z ich uczestnikami. Uczestnikami mogą być zarówno pracownicy (`User`) jak i klienci (`Client`). Pole `last_read_at` pozwala obliczać liczbę nieprzeczytanych wiadomości per uczestnik.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator udziału w rozmowie.
conversation_id	bigint	FK	Rozmowa → `conversations.id`.
participable_type	varchar	POLY	Klasa uczestnika: `User` | `Client`.
participable_id	bigint	POLY	ID uczestnika.
last_read_at	timestamp	NULL	Czas ostatniego odczytania rozmowy przez tego uczestnika. Używany do obliczania badge z liczbą nieprzeczytanych: `messages WHERE created_at > last_read_at`.

RELACJE

belongsTo → conversations | morphTo → users / clients

messages

Warstwa 3

Wiadomości w rozmowie. Wysyłane przez pracowników lub klientów. Każda nowa wiadomość wyzwala broadcast przez Reverb → Echo do wszystkich uczestników online oraz tworzy wpisy w `notifications` dla tych offline. Może zawierać @wzmianki.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator wiadomości.
conversation_id	bigint	FK	Rozmowa, do której należy wiadomość → `conversations.id`.
sender_type	varchar	POLY	Klasa nadawcy: `User` | `Client`.
sender_id	bigint	POLY	ID nadawcy wiadomości.
body	text		Treść wiadomości. Może zawierać @wzmianki — parser tworzy wpisy w `mentions`.
created_at	timestamp		Data wysłania wiadomości. Używana do sortowania i obliczania nieprzeczytanych.

RELACJE

belongsTo → conversations | morphTo → users / clients (sender) | morphMany → mentions

mentions

Warstwa 3

Rejestr @wzmianek w wiadomościach, komentarzach ticketów i komentarzach dokumentów. Każde oznaczenie tworzy rekord i wyzwala powiadomienie typu `mention` do oznaczonej osoby.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator wzmianki.
mentionable_type	varchar	POLY	Klasa obiektu zawierającego wzmiankę: `Message` | `TicketComment` | `DocumentComment`.
mentionable_id	bigint	POLY	ID wiadomości lub komentarza zawierającego wzmiankę.
mentioned_type	varchar	POLY	Klasa oznaczonej osoby: `User` | `Client`.
mentioned_id	bigint	POLY	ID oznaczonej osoby.
created_at	timestamp		Data dodania wzmianki.

RELACJE

morphTo → **messages** / **ticket_comments** (mentionable)

morphTo → **users** / **clients** (mentioned)

activity_log

Warstwa 3 Spatie

Audit trail całego systemu — zarządzany przez Spatie Activity Log. Każda akcja admina i klienta (create, update, delete, status change) jest rejestrowana automatycznie przez Model Events lub ręcznie. Przechowuje diff (poprzednia vs nowa wartość) w polu `properties`.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator wpisu w logu.
log_name	varchar	NULL	Kategoria logu np. `default`, `auth`, `crm`. Umożliwia filtrowanie logów po obszarze systemu.
description	text		Opis akcji np. `created`, `updated`, `deleted`, `status_changed`.
subject_type	varchar	POLY NULL	Klasa modelu, na którym wykonano akcję np. Lead, `Ticket`, `Client`.
subject_id	bigint	POLY NULL	ID obiektu, na którym wykonano akcję.
causer_type	varchar	POLY NULL	Klasa wykonawcy akcji: `User` | `Client`. `null` = akcja systemowa.

KOLUMNA	TYP	FLAGI	OPIS
causer_id	bigint	POLY NULL	ID wykonawcy akcji.
properties	json	JSON NULL	Diff zmian: `{"old": {"status": "new"}, "attributes": {"status": "contacted"}}`. Kluczowe dla audytu — pozwala odtworzyć historię każdego rekordu.
created_at	timestamp		Data i czas wykonania akcji.

RELACJE

morphTo → dowolny model (subject)

morphTo → **users** / **clients** (causer)

slug_redirects

Warstwa 3

Archiwum starych slugów po ręcznej zmianie. Middleware sprawdza tę tabelę gdy zasób nie zostanie znaleziony po aktualnym slugu i zwraca przekierowanie 301 do nowego URL. Zapobiega martwym linkom po rebrandingu lub zmianach nazw.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator przekierowania.
sluggable_type	varchar	POLY	Klasa modelu, którego slug się zmienił: `Client` | `Project` | `Lead` | `Offer` | `Document` | `Audit`.
sluggable_id	bigint	POLY	ID rekordu, którego slug się zmienił.
old_slug	varchar		Stary slug — ten, który ma być przekierowany do nowego URL rekordu.
changed_by	bigint	FK	Użytkownik który dokonał zmiany slugu → `users.id`. Do celów audytu.
created_at	timestamp		Data zmiany slugu.

RELACJE

morphTo → dowolny sluggable model

belongsTo → **users** (changed_by)

seo_analyses

Warstwa 3

Wyniki analizy SEO projektu. Może być samodzielna (`audit_id = null`) lub składową audytu zbiorczego. Pipeline kolejkowy: CrawlWebsiteJob → FetchGSCDataJob + FetchGA4DataJob → AnalyzeKeywordsJob + AnalyzeSemanticsJob → GenerateAuditReportJob → NotifyAuditCompleteJob.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator analizy SEO.
project_id	bigint	FK	Projekt, którego strona jest analizowana → `projects.id`.
audit_id	bigint	FK NULL	Audyt nadrzędny → `audits.id`. `null` = samodzielna analiza widoczna w sekcji „Analizy SEO" projektu.
status	enum	ENUM	Stan przetwarzania: `pending` → `running` → `completed` | `failed`. Aktualizowany przez joby w kolejce.
score	int	NULL	Ogólny wynik SEO 0–100. Obliczany po zakończeniu analizy na podstawie liczby i wagi znalezionych problemów.
keywords_count	int		Liczba monitorowanych słów kluczowych wykrytych podczas analizy.
issues_count	int		Łączna liczba wykrytych problemów SEO (wszystkie severity).
gsc_connected	boolean	BOOL	Czy dane z Google Search Console były dostępne podczas analizy.
ga4_connected	boolean	BOOL	Czy dane z Google Analytics 4 były dostępne podczas analizy.
completed_at	timestamp	NULL	Data zakończenia analizy (ostatni job w pipeline).
created_at	timestamp		Data zlecenia analizy.

RELACJE

- belongsTo → `projects`
- belongsTo → `audits` (opcjonalnie)
- hasMany → `seo_issues`
- hasMany → `seo_keyword_ranks`
- hasMany → `seo_monthly_snapshots`
- hasMany → `seo_cwv_snapshots`
- hasMany → `seo_backlinks`
- hasMany → `seo_keyword_opportunities`
- hasMany → `seo_competitor_ranks`

seo_issues

Warstwa 3

Wykryte problemy SEO z konkretnych analiz. Każdy problem ma przypisany poziom krytyczności i opcjonalne przypisanie do pracownika. Może być powiązany z

poprzednią wersją tego samego problemu (do śledzenia regresji).

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator problemu SEO.
seo_analysis_id	bigint	FK	Analiza, podczas której wykryto problem → seo_analyses.id.
previous_issue_id	bigint	FK NULL	Problem z poprzedniej analizy, który odpowiada temu samemu błędowi → seo_issues.id. Umożliwia śledzenie czy problem jest nowy czy powraca.
type	varchar		Typ problemu np. missing_meta_description, broken_link, duplicate_h1, slow_page.
severity	enum	ENUM	Krytyczność problemu: critical — blokuje pozycjonowanie, warning — obniża wynik, info — do rozważenia.
url	varchar		URL strony, na której wykryto problem.
description	text		Szczegółowy opis problemu — co jest nie tak i dlaczego wpływa na SEO.
recommendation	text	NULL	Konkretna rekomendacja naprawy — generowana przez AI (Claude/OpenAI) lub predefiniowana per typ.
assigned_to	bigint	FK NULL	Pracownik odpowiedzialny za naprawę → users.id.
due_date	date	NULL	Termin naprawy problemu.
notes	text	NULL	Wewnętrzne notatki pracownika dotyczące naprawy.
fixed_at	timestamp	NULL	Data naprawy problemu. null = problem wciąż aktywny.

RELACJE

belongsTo → seo_analyses

belongsTo → seo_issues (previous_issue)

belongsTo → users (assigned_to)

seo_keyword_ranks

Warstwa 3 TimescaleDB

Pozycje słów kluczowych w czasie — hypertabela TimescaleDB (time-series). Dane z Google Search Console: pozycja, kliknięcia, wyświetlenia, CTR. Partycjonowana po czasie dla wydajnych zapytań zakresu dat i trendów pozycji.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator rekordu rankingu.
seo_analysis_id	bigint	FK	Analiza SEO, z której pochodzi ten ranking → seo_analyses.id.
keyword	varchar		Słowo kluczowe lub fraza np. „agencja SEO Warszawa".
position	int		Średnia pozycja w wynikach Google dla tego słowa w danym okresie (1 = najlepsza).
clicks	int		Liczba kliknięć z Google Search dla tego słowa w danym okresie.
impressions	int		Liczba wyświetleń strony w wynikach dla tego słowa.
ctr	decimal		Click-Through Rate = clicks / impressions. Wyrażony jako ułamek dziesiętny (0.05 = 5%).
url	varchar		URL strony rankującej dla tego słowa kluczowego.
device	enum	ENUM	Urządzenie: desktop | mobile | tablet. GSC raportuje osobno dla każdego urządzenia.
time	timestamp	TIME COLUMN	Kolumna czasu hypertabeli TimescaleDB — punkt startowy partycji czasowej. Używana do zapytań time_bucket() i wykresów trendów.

RELACJE

belongsTo → seo_analyses

seo_monthly_snapshots

Warstwa 3

Zagregowane miesięczne podsumowania wyników SEO — dane z GSC i GA4. Przechowuje pre-agregowane metryki co eliminuje potrzebę drogi agregacji w locie przy generowaniu raportów miesięcznych dla klienta.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator snapshotu.

KOLUMNA	TYP	FLAGI	OPIS
seo_analysis_id	bigint	FK	Analiza SEO, z której pochodzi snapshot → seo_analyses.id.
period_start	date		Pierwszy dzień okresu (np. 2026-01-01).
period_end	date		Ostatni dzień okresu (np. 2026-01-31).
organic_sessions	int		Sesje z ruchu organicznego (Google) — z GA4.
organic_conversions	int		Konwersje z ruchu organicznego — z GA4.
avg_ctr	decimal		Średni CTR w całym miesiącu — z GSC.
avg_position	decimal		Średnia pozycja we wszystkich słowach kluczowych w miesiącu — z GSC.
top3_count	int		Liczba słów kluczowych w TOP 3 Google.
top10_count	int		Liczba słów kluczowych w TOP 10 Google.
top50_count	int		Liczba słów kluczowych w TOP 50 Google.
impressions	int		Łączna liczba wyświetleń w GSC w miesiącu.
clicks	int		Łączna liczba kliknięć z GSC w miesiącu.
new_backlinks	int		Nowe backlinki pozyskane w miesiącu.
lost_backlinks	int		Backlinki utracone w miesiącu.
referring_domains	int		Łączna liczba unikalnych domen odsyłających na koniec miesiąca.
created_at	timestamp		Data wygenerowania snapshotu.

RELACJE

belongsTo → seo_analyses

seo_cwv_snapshots

Warstwa 3 TimescaleDB

Pomiary Core Web Vitals per URL i urządzenie — hypertabela TimescaleDB. Dane z PageSpeed Insights API i GSC Field Data. Pozwala śledzić trendy wydajności strony w czasie: LCP, CLS, INP (wskaźniki Google page experience).

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator pomiaru.
seo_analysis_id	bigint	FK	Analiza SEO → seo_analyses.id.

KOLUMNA	TYP	FLAGI	OPIS
url	varchar		Mierzony URL strony.
lcp_ms	int	NULL	Largest Contentful Paint w milisekundach. Google: <2500ms dobry, >4000ms zły.
cls_score	decimal	NULL	Cumulative Layout Shift — wskaźnik wizualnej stabilności. Google: <0.1 dobry, >0.25 zły.
inp_ms	int	NULL	Interaction to Next Paint w milisekundach (następca FID). Google: <200ms dobry, >500ms zły.
fcp_ms	int	NULL	First Contentful Paint w milisekundach.
ttfb_ms	int	NULL	Time to First Byte w milisekundach — czas odpowiedzi serwera.
performance_score	int	NULL	Łączny wynik Performance z PageSpeed Insights (0–100).
device	enum	ENUM	Urządzenie pomiaru: `mobile` | `desktop`. PSI raportuje osobno.
source	enum	ENUM	Źródło danych: `pagespeed` — lab data z PSI API, `gsc_field` — field data z GSC (rzeczywiste dane użytkowników).
measured_at	timestamp	TIME COLUMN	Kolumna czasu hypertabeli TimescaleDB — czas wykonania pomiaru.

RELACJE

belongsTo → `seo_analyses`

seo_backlinks

Warstwa 3

Backlinki — linki zewnętrzne wskazujące na stronę klienta. Śledzenie profilu linków: nowe pozyskane, utracone, toxic links. Źródło danych: zewnętrzne API (Ahrefs/Semrush) lub własny crawler.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator backlinku.
seo_analysis_id	bigint	FK	Analiza SEO → `seo_analyses.id`.
referring_domain	varchar		Domena odsyłająca (np. `example.com`).
target_url	varchar		URL strony klienta, do której prowadzi link.
anchor_text	varchar	NULL	Tekst kotwicy linku. Ważny dla SEO — powinien zawierać słowa kluczowe.

KOLUMNA	TYP	FLAGI	OPIS
domain_rating	int	NULL	Autorytet domeny odsyłającej (0–100, Ahrefs DR lub podobna metryka). Im wyższy, tym bardziej wartościowy link.
is_toxic	boolean	BOOL	Czy link jest uznany za szkodliwy (spam, PBN, niska jakość). Linki toksyczne powinny być dodane do Google Disavow.
first_seen_at	date	NULL	Data pierwszego wykrycia linku.
last_seen_at	date	NULL	Data ostatniej weryfikacji że link nadal istnieje.
lost_at	date	NULL	Data utraty linku. `null` = link nadal aktywny.
created_at	timestamp		Data dodania rekordu do bazy.

RELACJE

belongsTo → **seo_analyses**

seo_competitors

Warstwa 3

Lista konkurentów monitorowanych dla danego projektu. Competitor to domena, której pozycje śledzić się w tych samych słowach kluczowych co klient. Używana do analizy porównawczej (gap analysis).

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator konkurenta.
project_id	bigint	FK	Projekt, dla którego monitorowany jest ten konkurent → `projects.id`.
domain	varchar		Domena konkurenta np. `konkurent.pl`.
label	varchar	NULL	Opcjonalna etykieta wyświetlana w UI np. „Główny konkurent" lub nazwa firmy.
is_active	boolean	BOOL	Czy konkurent jest aktualnie monitorowany. `false` = wyłączony z kolejnych analiz bez usuwania historii.
added_at	timestamp		Data dodania konkurenta do monitorowania.

RELACJE

belongsTo → **projects**

hasMany → **seo_competitor_ranks**

seo_competitor_ranks

Warstwa 3 TimescaleDB

Pozycje konkurentów dla tych samych słów kluczowych — hypertabela TimescaleDB. Pozwala tworzyć porównawcze wykresy: klient vs konkurenci w czasie. Używana w analizie gap keywords.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator rekordu.
competitor_id	bigint	FK	Konkurent → `seo_competitors.id`.
seo_analysis_id	bigint	FK	Analiza SEO, podczas której pobrano dane → `seo_analyses.id`.
keyword	varchar		Słowo kluczowe — powinno pokrywać się ze słowami z `seo_keyword_ranks` aby umożliwić porównanie.
position	int	NULL	Pozycja konkurenta dla słowa kluczowego. `null` = brak w TOP 100.
url	varchar	NULL	URL strony konkurenta rankującej dla tego słowa.
time	timestamp	TIME COLUMN	Kolumna czasu hypertabeli TimescaleDB.

RELACJE

belongsTo → seo_competitors

belongsTo → seo_analyses

seo_keyword_opportunities

Warstwa 3

Szanse słów kluczowych — frazy o dobrym wolumenie, na które klient jeszcze nie rankuje lub rankuje słabo. Generowane przez AI na podstawie analizy gap względem konkurentów i danych z GSC.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator szansy.
seo_analysis_id	bigint	FK	Analiza SEO → `seo_analyses.id`.
keyword	varchar		Fraza kluczowa będąca szansą.
search_volume	int		Miesięczna liczba wyszukiwań frazy w Google (z zewnętrznego API).
difficulty	int	NULL	Trudność pozycjonowania frazy (0–100). Im wyższy, tym trudniej wyprzedzić konkurencję.
competitor_positions	json	JSON NULL	Pozycje konkurentów dla tej frazy: `{"konkurent.pl": 3, "inny.pl": 7}`. Kontekst do oceny szansy.
has_dedicated_page	boolean	BOOL	Czy klient ma już stronę zoptymalizowaną pod tę frazę. `false` = trzeba stworzyć nową stronę

KOLUMNA	TYP	FLAGI	OPIS
			lub treść.
recommendation	text	NULL	Rekomendacja AI: co zrobić żeby zaatakować tę frazę — nowa podstrona, rozszerzenie treści, linkowanie wewnętrzne.
created_at	timestamp		Data wykrycia szansy.

RELACJE

belongsTo → **seo_analyses**

audits

Warstwa 3

Audyty zbiorcze — byt nadrzędny grupujący analizy różnych typów w jeden raport. Audyt typu `full` uruchamia wszystkie składowe (SEO, techniczny, UX, security). Każda składowa może być też uruchomiona samodzielnie jako osobna analiza.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator audytu.
project_id	bigint	FK	Projekt, dla którego przeprowadzany jest audyt → `projects.id`.
type	enum	ENUM	Zakres audytu: `seo` | `technical` | `ux` | `security` | `full` (wszystkie typy naraz).
status	enum	ENUM	Stan audytu: `pending` → `running` → `completed` | `failed`.
overall_score	int	NULL	Zagregowany wynik audytu 0–100. Obliczany jako ważona średnia wyników z `audit_scores` per kategoria.
slug	varchar		Identyfikator URL audytu, unikalny per `project_id`. Immutable — Spatie Sluggable.
started_at	timestamp	NULL	Data uruchomienia pierwszego joba w pipeline audytu.
completed_at	timestamp	NULL	Data zakończenia ostatniego joba w pipeline. Wyzwala `NotifyAuditCompleteJob`.
created_by	bigint	FK	Pracownik który zlecił audyt → `users.id`.

RELACJE

belongsTo → **projects**

belongsTo → **users** (created_by)

hasMany → **seo_analyses**

hasMany → **audit_issues**

hasMany → **audit_scores**

audit_issues

Warstwa 3

Problemy wykryte podczas audytu zbiorczego — szerszy zakres niż `seo_issues`, obejmuje UX, security, performance i treść. Każdy problem ma kategorię, krytyczność i rekomendację naprawy.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator problemu.
audit_id	bigint	FK	Audyt, podczas którego wykryto problem → `audits.id`.
category	enum	ENUM	Obszar problemu: ux | `security` | `performance` | `technical` | `content` | `seo`. Odpowiada kategoriom w `audit_scores`.
type	varchar		Szczegółowy typ problemu np. `no_https`, `missing_alt_text`, `sql_injection_risk`.
severity	enum	ENUM	Krytyczność: `critical` | `warning` | `info`.
url	varchar	NULL	URL strony z problemem. `null` = problem dotyczy całej witryny.
description	text		Opis problemu.
recommendation	text	NULL	Konkretna rekomendacja naprawy — może być generowana przez AI.
assigned_to	bigint	FK NULL	Pracownik odpowiedzialny za naprawę → `users.id`.
due_date	date	NULL	Termin naprawy.
fixed_at	timestamp	NULL	Data naprawy problemu. `null` = problem aktywny.
created_at	timestamp		Data wykrycia problemu.

RELACJE

belongsTo → **audits**

belongsTo → **users** (assigned_to)

audit_scores

Warstwa 3

Wyniki punktowe audytu per kategoria. Każdy audyt ma osobny wynik dla SEO, performance, UX, security i content. Używane do wykresów radar/gauge w raporcie audytu i obliczania `audits.overall_score`.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator wyniku.

KOLUMNA	TYP	FLAGI	OPIS
audit_id	bigint	FK	Audyt, do którego należy wynik → audits.id.
category	enum	ENUM	Kategoria oceniana: seo | performance | ux | security | content. Para (audit_id, category) powinna być unikalna.
score	int		Wynik kategorii 0–100. Obliczany na podstawie liczby i wagi problemów (audit_issues) w tej kategorii.
details	json	JSON NULL	Szczegóły składowe wyniku: {"critical": 2, "warnings": 5, "info": 12, "passed": 43}. Używane do breakdown w UI.
measured_at	timestamp		Data i czas obliczenia wyniku.

RELACJE

belongsTo → audits

audit_reports

Warstwa 3

Raporty PDF generowane po zakończeniu audytu. Jeden raport per audyt (unique). Zawiera publiczny token umożliwiający klientowi przeglądanie raportu bez logowania. Strategist może dodać ręczny komentarz przed wysyłką. Śledzony jest moment i wykonawca wysyłki do klienta.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator raportu.
audit_id	bigint	FK	Audyt do którego należy raport → audits.id. Unique — jeden raport per audyt.
report_type	enum	ENUM	Typ raportu: seo, ux, security, full (wszystkie komponenty połączone).
pdf_path	varchar	NULL	Ścieżka do pliku PDF w S3/MinIO (np. reports/2026/03/audit-abc123.pdf). null = PDF jeszcze nie wygenerowany.
pdf_generated_at	timestamp	NULL	Data i czas wygenerowania PDF przez Browsershot (headless Chromium).
public_token	uuid	NULL	Token UUID do publicznego podglądu raportu bez logowania. Generowany przy wysyłce do klienta. Link: /raport/{public_token}.
public_expires_at	timestamp	NULL	Opcjonalna data wygaśnięcia publicznego linku. null = link aktywny bezterminowo.
strategist_comment	text	NULL	Ręczny komentarz stratega dodawany przed wysyłką do klienta. Pojawia się jako wstęp w raporcie. Może wyjaśniać priorytety i kontekst.

KOLUMNA	TYP	FLAGI	OPIS
sent_to_client_at	timestamp	NULL	Data i czas wysyłki raportu do klienta (email z linkiem).
sent_by	bigint	FK NULL	Pracownik który wysłał raport → users.id.

RELACJE

belongsTo → **audits** (unique — 1:1)

belongsTo → **users** (sent_by)

ux_audit_results

Warstwa 3

Wyniki automatycznego audytu UX/dostępności przeprowadzonego przez axe-core lub WAVE API. Jeden rekord per strona per audyt. Zawiera pełny raport naruszeń WCAG, liczbę problemów kontrastowych, brakujących atrybutów alt, oraz zagregowane oceny nawigacji klawiaturowej i responsywności.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator wyniku UX.
audit_id	bigint	FK	Audyt do którego należy wynik → audits.id.
page_url	varchar		URL przeskanowanej strony.
wcag_violations	json	JSON	Pełny output axe-core: lista naruszeń WCAG z ID, opisem, impactem i selektorami CSS elementów naruszających.
contrast_issues	int		Liczba problemów kontrastowych (kolor tekstu vs tło) niezgodnych z WCAG AA.
missing_alt_count	int		Liczba obrazów bez atrybutu alt.
keyboard_nav_score	tinyint		Ocena nawigacji klawiaturowej (1–10). Opiera się na analizie focus order, pułapek focusu i widocznych outline'ów.
mobile_usability_score	tinyint		Ocena użyteczności mobilnej (1–10). Opiera się na wielkości elementów kliknięcia, viewport meta i responsywności layoutu.
overall_score	decimal(4,2)		Zagregowana ocena UX strony (0.00–100.00). Wchodzi do scoring audytu (audit_scores).
raw_report	json	JSON NULL	Surowy raport z axe-core/WAVE — pełne dane do ewentualnego re-parsowania lub eksportu.

KOLUMNA	TYP	FLAGI	OPIS
analyzed_at	timestamp		Data i czas przeprowadzenia analizy.

RELACJE

belongsTo → **audits**

security_audit_results

Warstwa 3

Wyniki automatycznego audytu bezpieczeństwa HTTP — nagłówki bezpieczeństwa, konfiguracja SSL/TLS. Dane pobierane przez securityheaders.com API i własny skaner. Jeden rekord per URL per audyt.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator wyniku security.
audit_id	bigint	FK	Audyt do którego należy wynik → `audits.id`.
target_url	varchar		URL przeskanowanej domeny/strony.
https_enabled	boolean		Czy strona serwowana przez HTTPS. Podstawowy wymóg bezpieczeństwa.
hsts_enabled	boolean		Czy nagłówek `Strict-Transport-Security` jest ustawiony poprawnie.
csp_score	tinyint		Ocena Content Security Policy (1–10). 10 = restrykcyjna polityka bez `unsafe-inline`.
headers_score	tinyint		Ogólna ocena nagłówków HTTP bezpieczeństwa (1–10) bazowana na securityheaders.com.
ssl_grade	varchar(4)	NULL	Ocena konfiguracji SSL/TLS wg Qualys SSL Labs: A+, A, B, C, F.
ssl_expires_at	date	NULL	Data wygaśnięcia certyfikatu SSL. Używana do alertów o zbliżającym się wygaśnięciu.
vulnerabilities	json	JSON NULL	Lista znalezionych podatności: `[{"type": "missing_csp", "severity": "high", "description": "..."}]`.
overall_score	decimal(4,2)		Zagregowana ocena bezpieczeństwa (0.00–100.00). Wchodzi do scoring audytu.
raw_report	json	JSON NULL	Surowy raport z zewnętrznych API do ewentualnego re-parsowania.
analyzed_at	timestamp		Data i czas przeprowadzenia analizy.

RELACJE

belongsTo → **audits**

block_templates

Warstwa 3

Predefiniowane szablony bloków TipTap (~20 typów). Każdy szablon zawiera strukturę JSON dokumentu TipTap z wypełnionymi zmiennymi do podstawienia. Używane przy tworzeniu ofert, dokumentów projektowych i treści SEO. Szablony mogą być globalne lub customizowane per projekt.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator szablonu.
slug	varchar		Unikalny identyfikator tekstowy szablonu (unique). Używany w kodzie do referencji np. `intro-problem-solution`.
name	varchar		Czytelna nazwa szablonu np. "Wprowadzenie + Problem + Rozwiązanie".
category	enum	ENUM	Kategoria bloku: `intro`, `problem`, `solution`, `cta`, `testimonial`, `faq`, `pricing`, `about`, `team`, `portfolio` i inne.
description	text	NULL	Opis kiedy i jak używać szablonu. Pomaga użytkownikowi wybrać odpowiedni blok.
content_json	json	JSON	Struktura bloku w formacie TipTap JSON ze zmiennymi do podstawienia np. `{{company_name}}`, `{{main_benefit}}`.
variables	json	JSON NULL	Lista zmiennych szablonu: `[{"name": "company_name", "description": "Nazwa firmy klienta", "placeholder": "np. Acme Sp. z o.o."}]`.
is_active	boolean		Czy szablon jest aktywny i dostępny do wyboru. `false` = ukryty (deprecated ale zachowany dla starych dokumentów).

RELACJE

hasMany → **block_template_scores**

block_template_scores

Warstwa 3

Scoring efektywności szablonów bloków — ocena AI i dane o częstotliwości użycia. Może być globalny (bez powiązanego projektu) lub specyficzny dla projektu (na podstawie wyników tego klienta). Pozwala AI rekomendować najlepsze bloki.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator rekordu scoringu.
template_id	bigint	FK	Szablon bloku → `block_templates.id`.
project_id	bigint	FK NULL	Projekt dla którego dotyczy scoring. `null` = scoring globalny (dla wszystkich projektów).
score	decimal(4,2)		Ocena efektywności szablonu (0.00–100.00).
usage_count	int		Liczba użyć szablonu (per projekt lub globalnie). Używana do priorytetyzacji rekomendacji.
ai_feedback	text	NULL	Feedback AI na temat efektywności szablonu — dlaczego działa dobrze lub co można poprawić.
scored_at	timestamp		Data i czas ostatniego przeliczenia scoringu.

RELACJE

belongsTo → `block_templates`

belongsTo → `projects` (opcjonalnie)

knowledge_facts

Warstwa 3

Fakty Bazy Wiedzy — atomowe informacje o kliencie, projekcie lub domenie. Każdy fakt ma typ (decyzja, wykluczenie, wymaganie itp.) oraz wektor embeddingu (1536-dim, pgvector) umożliwiający semantic search. Fakty mogą być globalne (np. zasady agencji) lub per klient. Przeszukiwanie przez IVFFlat index — nearest-neighbor w czasie <50ms.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator faktu.
client_id	bigint	FK NULL	Klient do którego należy fakt. `null` = fakt globalny (dotyczy wszystkich klientów lub agencji).
type	enum	ENUM	Typ faktu: `decision` — podjęta decyzja, `exclusion` — wykluczenie/czego nie robić, `requirement` — wymaganie techniczne, `preference` — preferencja klienta, `constraint` — ograniczenie.
content	text		Treść faktu w języku naturalnym np. "Klient nie chce używać WordPress — wymaga React SPA".

KOLUMNA	TYP	FLAGI	OPIS
confidence	decimal(3,2)		Pewność faktu (0.00–1.00). 1.00 = fakt potwierdzony, niższe wartości = fakt inferencyjny lub niepewny.
embedding	vector(1536)		Wektor embeddingu (OpenAI text-embedding-3-small lub Claude). Używany do semantic search przez pgvector IVFFlat index.
source_type	varchar	POLY	Typ źródła skąd pochodzi fakt: `Lead`, `Project`, `Ticket`, `Manual` (ręcznie wprowadzony).
source_id	bigint	POLY NULL	ID rekordu źródłowego (morphs z source_type). `null` dla `Manual`.
is_active	boolean		Czy fakt jest aktywny. `false` = fakt zdezaktualizowany (soft delete).
created_by	bigint	FK NULL	Użytkownik który stworzył fakt. `null` = generowany przez AI.

RELACJE

belongsTo → **clients** (opcjonalnie)

morphsTo → **leads** | **projects** | **tickets**

knowledge_entities

Warstwa 3

Encje Bazy Wiedzy — nazwane byty (technologie, firmy, osoby, produkty, koncepty). Każda encja ma wektor embeddingu do semantic search i metadane JSON. Encje łączą się ze sobą przez `knowledge_entity_relations` tworząc graf wiedzy.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator encji.
client_id	bigint	FK NULL	Klient do którego należy encja. `null` = encja globalna.
name	varchar		Nazwa encji np. "React", "Google Search Console", "Acme S.A.", "Jan Kowalski".
entity_type	varchar		Typ encji np. `technology`, `company`, `person`, `product`, `concept`, `tool`.
description	text	NULL	Opis encji w kontekście projektu/klienta.
embedding	vector(1536)		Wektor embeddingu encji (1536-dim). Używany do semantic search i znajdowania podobnych encji.
metadata	json	JSON NULL	Dowolne dodatkowe atrybuty encji np. URL, wersja, alias, powiązane słowa kluczowe.

RELACJE

belongsTo → **clients** (opcjonalnie)

hasMany → **knowledge_entity_relations** (subject)

hasMany → `knowledge_entity_relations` (object)

knowledge_entity_relations

Warstwa 3

Relacje między encjami Bazy Wiedzy — trójki (subject, predicate, object). Tworzą graf wiedzy: np. "React *is a* JavaScript Framework", "Projekt X *requires* PHP 8.3", "Klient Y *dislikes* WordPress". Każda relacja ma współczynnik pewności.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator relacji.
subject_id	bigint	FK	Podmiot relacji → `knowledge_entities.id`.
predicate	varchar		Predykat relacji np. `is_a`, `has`, `requires`, `conflicts_with`, `uses`, `replaces`, `dislikes`.
object_id	bigint	FK	Dopełnienie relacji → `knowledge_entities.id`.
confidence	decimal(3,2)		Pewność relacji (0.00–1.00). Relacje inferencyjne mają niższe wartości.
created_at	timestamp		Data i czas utworzenia relacji.

RELACJE

belongsTo → `knowledge_entities` (subject)

belongsTo → `knowledge_entities` (object)

knowledge_items

Warstwa 3

Hierarchiczne drzewo wiedzy — elementy z możliwością zagnieżdżenia (self-referential przez `parent_id`). Każdy item ma status cyklu życia (proponowany → potwierdzony → odrzucony) i wektor embeddingu. Używane do budowania strukturyzowanej bazy wiedzy per klient.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator elementu wiedzy.
client_id	bigint	FK NULL	Klient do którego należy element. `null` = element globalny.
parent_id	bigint	FK NULL	Rodzic elementu w hierarchii → `knowledge_items.id`. `null` = element główny (root).
title	varchar		Tytuł elementu wiedzy.
content	text	NULL	Treść elementu — rozwinięty opis, instrukcje lub kontekst.

KOLUMNA	TYP	FLAGI	OPIS
item_type	varchar		Typ elementu np. category, guideline, example, template, note.
status	enum	ENUM	proposed — proponowany przez AI lub użytkownika, confirmed — zatwierdzony przez managera, rejected — odrzucony.
embedding	vector(1536)		Wektor embeddingu (1536-dim). Indeksowany IVFFlat dla semantic search w Bazie Wiedzy.
created_by	bigint	FK NULL	Użytkownik który stworzył element. null = generowany przez AI.
confirmed_by	bigint	FK NULL	Manager który zatwierdził element → users.id. null = niezatwierdzony.

RELACJE

- belongsTo → **clients** (opcjonalnie)
- belongsTo → **knowledge_items** (parent — self-referential)
- hasMany → **knowledge_items** (children)

agent_knowledge

Warstwa 3

Globalna pamięć robocza agentów AI (np. "Kasia"). Przechowuje key-value pairs z wektorem embeddingu umożliwiającym semantic retrieval. Agent może odczytać relevantne wpisy do swojego kontekstu przed każdą rozmową. Niezależna od per-klientowej Bazy Wiedzy.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator wpisu wiedzy agenta.
agent_id	varchar		Identyfikator agenta np. kasia, seo-analyst. Umożliwia izolację wiedzy per agent.
key	varchar		Klucz wpisu np. agency_tone_of_voice, discovery_best_practices.
value	text		Wartość wpisu — instrukcje, wytyczne lub kontekst dla agenta w języku naturalnym.
embedding	vector(1536)		Wektor embeddingu wartości. Umożliwia semantic retrieval — agent wyszukuje relevantne wpisy do swojego bieżącego kontekstu.
updated_at	timestamp		Data i czas ostatniej aktualizacji wpisu.

RELACJE

- Brak relacji FK — globalna tabela agentów

ai_visibility_scores

Warstwa 3

8-czynnikowy scoring widoczności AI projektu/klienta w modelach językowych. Mierzy jak bardzo treści klienta są "AI-friendly" — cytowalne, strukturalne, autorytatywne. Historia scoringów zachowywana — jeden rekord per pomiar (może być wiele per projekt). Wyniki wyświetlane na dashboardzie jako trend w czasie.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator pomiaru.
project_id	bigint	FK	Projekt dla którego przeprowadzono pomiar → projects.id.
client_id	bigint	FK	Klient (multi-tenancy) → clients.id.
brand_mentions_score	tinyint		Wskaźnik wzmiankowania marki w sieci (1–10). Opiera się na external link profile i NAP consistency.
content_authority_score	tinyint		Autorytet treści (1–10). Głębokość, aktualność, źródła, eksperckość.
structured_data_score	tinyint		Jakość danych strukturalnych Schema.org (1–10). Ułatwia LLM parsowanie treści.
page_speed_score	tinyint		Prędkość ładowania strony (1–10). Opiera się na Core Web Vitals LCP/INP/CLS.
mobile_score	tinyint		Responsywność i użyteczność mobilna (1–10).
e_e_a_t_score	tinyint		E-E-A-T score (Experience, Expertise, Authoritativeness, Trustworthiness) (1–10).
llm_readability_score	tinyint		Czytelność treści dla modeli językowych (1–10). Klarowność, brak redundancji, logiczna struktura.
citation_potential_score	tinyint		Potencjał cytowania przez LLM jako źródło (1–10). Oryginalność danych, statystyki, autorytety.
overall_score	decimal(4,2)		Ważona średnia wszystkich 8 czynników. Zakres 1.00–10.00. Główna metryka widoczności AI.
ai_reasoning	text	NULL	Uzasadnienie i rekomendacje poprawy wygenerowane przez AI.
scored_at	timestamp		Data i czas przeprowadzenia scoringu.

RELACJE

belongsTo → projects

belongsTo → clients

seo_workshop_steps

Warstwa 3

Kroki 4-etapowego SEO Workshop per projekt. Każdy etap produkuje output (dane wejściowe → dane wyjściowe w JSON). Krok 4 (deploy) automatycznie wysyła treść na WordPress przez WP REST API. Każdy projekt może mieć wiele przejść przez workshop (nowe cykle treści).

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator kroku.
project_id	bigint	FK	Projekt do którego należy krok SEO Workshop → `projects.id`.
step_number	tinyint		Numer kroku (1–4): 1=analiza, 2=strategia, 3=treść, 4=deploy.
step_type	enum	ENUM	`analysis` — analiza słów kluczowych i konkurencji, `strategy` — plan contentowy, `content` — generowanie treści AI, `deploy` — publikacja na WordPress przez WP REST API.
status	enum	ENUM	`pending`, `in_progress`, `completed`, `failed`.
input_data	json	JSON NULL	Dane wejściowe kroku. Dla kroku 2: wyniki analizy z kroku 1. Dla kroku 3: strategia z kroku 2. Dla deploy: treść z kroku 3.
output_data	json	JSON NULL	Dane wyjściowe kroku. Dla deploy: zawiera `wp_post_id`, URL i timestamp publikacji.
wp_post_id	int	NULL	ID posta WordPress po pomyślnym deployu przez WP REST API (`POST /wp-json/wp/v2/posts`). `null` = niezdeployowane lub nie krok deploy.
wp_deployed_at	timestamp	NULL	Data i czas publikacji na WordPress.
completed_at	timestamp	NULL	Data i czas zakończenia kroku.

RELACJE

belongsTo → **projects**

work_logs

Warstwa 3

Dziennik pracy per projekt — rejestracja czasu pracy pracowników z podziałem na kategorie. Umożliwia raportowanie czasu per klient, per kategoria i per pracownik. Podstawa do fakturowania i analizy rentowności projektu.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator wpisu dziennika.

KOLUMNA	TYP	FLAGI	OPIS
project_id	bigint	FK	Projekt do którego należy wpis → projects.id.
user_id	bigint	FK	Pracownik który wykonał pracę → users.id.
category	enum	ENUM	Kategoria pracy: development, design, seo, copywriting, meetings, admin, other.
description	text		Opis wykonanej pracy — co konkretnie zostało zrobione.
duration_minutes	int		Czas pracy w minutach. Wyświetlany w UI jako godziny:minuty.
logged_at	date		Data wykonania pracy (nie data wpisu — pozwala na retrospektywne logowanie).
created_at	timestamp		Data i czas dodania wpisu do systemu.

RELACJE

belongsTo → projects

belongsTo → users

work_plans

Warstwa 3

Plany pracy per projekt — tygodniowe lub miesięczne harmonogramy. Plan może być przypisany do konkretnego pracownika lub być planem całego zespołu. Każdy plan zawiera listę elementów (work_plan_items) do wykonania w danym okresie.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator planu.
project_id	bigint	FK	Projekt do którego należy plan → projects.id.
user_id	bigint	FK NULL	Pracownik do którego przypisany jest plan. null = plan całego zespołu.
period_type	enum	ENUM	weekly — tygodniowy (pon-nd), monthly — miesięczny.
period_start	date		Data początkowa okresu planu.
period_end	date		Data końcowa okresu planu.
status	enum	ENUM	draft — w przygotowaniu, active — bieżący aktywny plan, completed — zakończony okres.

RELACJE

belongsTo → projects

belongsTo → users (opcjonalnie)

hasMany → work_plan_items

work_plan_items

Warstwa 3

Pojedyncze zadanie w planie pracy. Zawiera szacowany i rzeczywisty czas, kategorię, przypisanego wykonawcę i termin. Statusy pozwalają śledzić postęp realizacji planu w trakcie okresu.

KOLUMNA	TYP	FLAGI	OPIS
id	bigint	PK	Unikalny identyfikator elementu planu.
plan_id	bigint	FK	Plan do którego należy element → work_plans.id.
title	varchar		Nazwa zadania do wykonania.
description	text	NULL	Opcjonalny szczegółowy opis zadania.
category	enum	ENUM	Kategoria taka sama jak w work_logs: development, design, seo, copywriting, meetings, admin, other.
estimated_hours	decimal(5,2)		Szacowany czas realizacji w godzinach.
actual_hours	decimal(5,2)	NULL	Rzeczywisty czas realizacji w godzinach (uzupełniany po zakończeniu). null = niezakończone.
priority	int		Kolejność wyświetlania i ważność zadania (niższy = ważniejszy). Umożliwia drag & drop sortowanie.
status	enum	ENUM	todo — do zrobienia, in_progress — w trakcie, done — ukończone, cancelled — anulowane.
assigned_to	bigint	FK NULL	Pracownik przypisany do zadania → users.id. null = nieprzypisane.
due_at	date	NULL	Opcjonalny termin realizacji zadania w ramach okresu planu.

RELACJE

- belongsTo → work_plans
- belongsTo → users (assigned_to)